

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Using Syntax to Structure a Program



Learning Outcome:

Exchange information about logic, its principles, and methods in simple spoken and written situations.

Ask your partner:

- Do you consider appropriate to organize commands when programming? Why?
- What are the most important aspects to execute codes of a computer program? Why?

Can Do Statements:

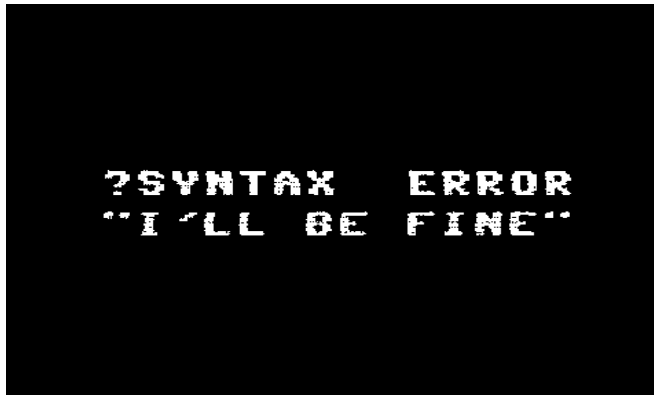
- I can talk about the concept of syntax in programming language.
- I can present in an easy way concepts like statements, variables, and keywords.
- I can discuss an example to understand the concept of controlling workflows.





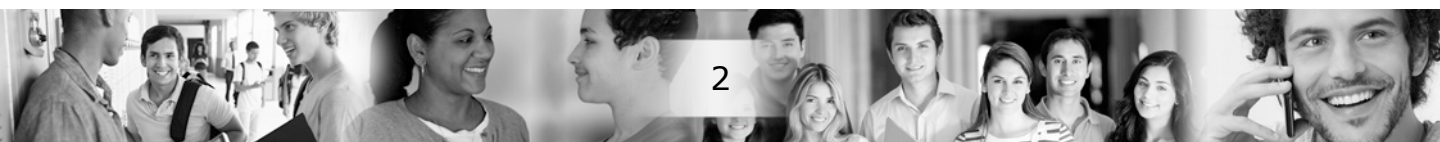
Let's Explore!

A student talks to his teacher after class.



Instructions: Read the text below.

- Peter:** Mr. Kumal can I talk to you? I want to make sure I am working on my project using the correct information.
- Mr. Kumal:** How can I help you?
- Peter:** First, you mentioned that when we are programming organization of words and symbols make any programming language usable, so we have to use programming comments.
- Mr. Kumal:** Exactly, continue please.
- Peter:** You mentioned syntax is a set of rules that can indicate the appropriate arrangement of words and symbols to structure a computer program. Thus, programmers are always paying attention to logic and syntax because any minor error can represent a mess in execution.
- Mr. Kumal:** Yes, why don't we go straight to the point?
- Peter:** Ok teacher. Can you tell me about statements, variables and keywords?
- Mr. Kumal:** Yes, syntax includes all of them; a statement is a simple unit of code established as an execution path. A variable can be a sign or a symbol that stores values like strings, numbers or Boolean values indicating an operation to perform. And the keywords are reserved commands with a specific meaning, when programmers are writing commands such as "print", "import" or "export", in fact they are using keywords.
- Peter:** Teacher, why don't we discuss about controlling workflow?
- Mr. Kumal:** Yes, let me give you an example; programs are executed following steps but, what can you do when you need to skip a step? Imagine you go to an ATM machine to withdraw \$100, if you have the correct balance, the machine gives you the money; but, if your balance is less than the requested amount, for sure you will see the message: "insufficient balance". That is the workflow path which is indicating how the program can skip a part of the code when necessary.





Let's Practice!

Syntax in programming language

Instructions: Classify the sentences by writing the number of the sentence in the correct category.

1. We are integrating logarithms.
2. Syntax is defining how to use the symbols.
3. We are including rules and patterns in syntax.
4. Syntax is determining changes in language.
5. Syntax is indicating how to execute a program.

Aspects that are possible to find in syntax.	Aspects that are not possible to find in syntax.



Let's Practice!

Examples about controlling workflow

Instructions: Read the sentences. Write the missing words in the blank. Use: can process, indicate, can.

1. In general, the workflow _____ the execution of a series of statements.
2. Banks _____ determine the conditional statements to programm ATM machines.
3. When you withdraw money from your account, the workflow path _____ programming statements within the ATM



Extension Activity

Important Information About Programming Logic and Syntax

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for information about the "if-statement" and the "if-else statement".
- Describe how each statement works.
- Choose one of the statements and write an example.
- Make a short presentation in which you explain the example to your classmates and instructor.

Self-Assessment

Yes No

- I look for information about two kinds of statements.
- I describe how statements work.
- I write examples about statements.
- I share statements information and examples.

☐	☐
☐	☐
☐	☐
☐	☐



Target Words

- **usable** (adjective): that is available and possibly valuable
e.g. When we are programming organization of words and symbols make any programming language usable.
- **execution path** (noun): it is a route indicated as a reference to execute a programming code
e.g. A statement is a simple unit of code established as an execution path.
- **boolean value** (noun): values with only two possibilities: true or false
e.g. A variable can be a sign or a symbol that stores values like strings, numbers or Boolean values.
- **withdraw** (verb): to take something out of, ask for something, like money
e.g. If you go to an ATM machine to withdraw \$100 and you have the correct balance...
- **arrangement** (noun): an organization and settlement of elements
e.g. syntax is a set of rules that can indicate the appropriate arrangement of words and symbols.
- **balance** (noun): the amount of money in the bank
e.g. If your balance is less than the requested amount, for sure you will see the message: "insufficient balance".



Learning Points

Present Continuous Affirmative

You use the **present continuous** to talk about an action that is happening at the moment of speaking.

The **present continuous** is used to describe current actions and facts that are currently happening, not at the moment of speaking but nowadays.

Let's take a review of the structure of affirmative statements in present continuous, it is as follows:

Subject + aux verb "to be"(am, is, are) + verb -ing+ complement

Example:

*Jacob is **writing** the program's code.*

*They are **working** on a new code.*

Remember the conjugation of the verb to be , which is used as an auxiliary verb:

- *I **am arranging** syntax.*
- *You **are encoding** symbols.*
- *He **is creating** conditional statements.*
- *She **is working** in the workflow.*
- *It **is executing** if-else statements.*
- *We **are checking** possible variables.*
- *They **are defining** keywords.*

There is a rule to add -ing suffix at the end of the verb, every time you have a doubt on how to do it, check the following chart:

RULE TO ADD -ING	COMMON EXAMPLES
Verb ends in an E we remove the E and add ING .	Take / taking Make/making
Verb ends in a consonant + vowel + consonant , we double the final consonant and add ING .	Get / getting Program/ programming Plan / planning
Do not double the final consonant when the verb ends in W, X or Y or when the final syllable is not emphasized.	Fix/ fixing
Verb ends in IE we change it to YING .	Lie/ lying



Learning Points

Examples

- *I am working on my project using the correct information.*
- *When we are programming organization of words and symbols make any programming language usable.*
- *Programmers are always paying attention to logic and syntax.*
- *When programmers are writing commands such as "print", "import" or "export", in fact they are using keywords.*
- *That is the workflow path which is indicating how the program can skip a part of the code when necessary.*

Comments in Programming

You use **programming comments** to guide yourself in the code you are writing or to help others understand.

When programmers are writing codes it is very common to find some of them difficult to understand. Some codes can be very confusing for anyone trying to interpret those. In order to avoid errors and clarify the logic of the code, programmers have the option to include a programming comment or an **annotation**.

A programming comment is included within special characters to avoid confusing this with the code. There are two kinds of comments: the 'end of line comment' and the 'block comment'.

As the name implies an 'end of the line' comment comes at the end of a code line, like this:

```
max = 100; // using default size.
```

A 'block comment' has a terminator and it can be very short (less than one line) or very long, using several lines, like this:

```
/* check input for valid values and print error message for accounting if problems. */
```

